prompt-populism vs. PopBERT Expert Coders – Agreement Comparison

This notebook rates a sample of 200 PopBERT gold-standard texts with an LLM system prompt and compares the results against human expert annotators using the same metrics reported in the PopBERT paper (Fleiss' κ, Agreement %, pairwise F1).

Dimension mapping: - `people_score > 0` ⟶ **People-Centrism** (1/0) - `elite_score < 0` ⟶ **Anti-Elitism** (1/0)

The prompt does not cover Left/Right ideology, so those dimensions are compared using expert-coder baselines only.

1 — Setup & Imports

```
import json
import os
import time
from itertools import combinations
from pathlib import Path

import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import numpy as np
import pandas as pd
from openai import OpenAI
from sklearn.metrics import cohen_kappa_score, f1_score
from statsmodels.stats.inter_rater import fleiss_kappa, aggregate_raters

np.random.seed(42)
plt.rcParams.update({'figure.dpi': 150, 'savefig.dpi': 300})
```

2 — Load PopBERT Gold-Standard Data

```
df_raw = pd.read_csv("data/full.csv.zip")
print(f"Shape: {df_raw.shape}")
print(f"Columns: {list(df_raw.columns)}")
print(f"Coders:  {df_raw['username'].unique()}")
print(f"Unique texts: {df_raw['id'].nunique()}")
df_raw.head(6)
```

```
Shape: (43975, 7)
Columns: ['id', 'text', 'username', 'elite', 'centr', 'left', 'right']
Coders:  <StringArray>
['riedel', 'richter', 'grabsch', 'schadt', 'coudry']
Length: 5, dtype: str
Unique texts: 8795
```

	id	text	username	elite	centr	left	right
0	446633	Ihre willkürliche Zusammenstellung und Auflist…	riedel	True	False	False	False
1	446633	Ihre willkürliche Zusammenstellung und Auflist…	richter	False	False	False	False
2	446633	Ihre willkürliche Zusammenstellung und Auflist…	grabsch	False	False	False	False
3	119028	Einen ganz herzlichen Dank an diese Kolleginne…	schadt	False	False	False	False
4	119028	Einen ganz herzlichen Dank an diese Kolleginne…	coudry	False	False	False	False
5	119028	Einen ganz herzlichen Dank an diese Kolleginne…	grabsch	False	False	False	False

3 — Column Mapping & Gold-Standard Extraction

Long format: one row per (text × coder), boolean columns `elite`, `centr`, `left`, `right`. Pivot to wide matrices, derive gold standard via majority vote.

```python
TEXT_COL = "text"
ID_COL   = "id"

DIM_MAP = {
    "elite": "anti_elitism",
    "centr": "people_centrism",
    "left":  "left_ideology",
    "right": "right_ideology",
}

LABEL_DIMENSIONS = list(DIM_MAP.values())

# The two dimensions our prompt can evaluate
LLM_DIMENSIONS = ["anti_elitism", "people_centrism"]

DISPLAY_NAMES = {
```

```
    "anti_elitism":    "Anti-Elitism",
    "people_centrism": "People-Centrism",
    "left_ideology":   "Left-Wing Ideology",
    "right_ideology":  "Right-Wing Ideology",
}
```

3a — Pivot to coder matrices & majority-vote gold standard

```
gold_matrices = {}
gold_standard = {}

for csv_col, dim_name in DIM_MAP.items():
    wide = df_raw.pivot_table(
        index=ID_COL, columns="username", values=csv_col, aggfunc="first",
    ).astype(float)
    gold_matrices[dim_name] = wide
    gold_standard[dim_name] = (wide.mean(axis=1) >= 0.5).astype(int)

texts = df_raw.drop_duplicates(subset=ID_COL).set_index(ID_COL)[[TEXT_COL]]

print(f"Unique texts: {len(texts)}")
print(f"Coders: {list(gold_matrices['anti_elitism'].columns)}")
for dim_name in LABEL_DIMENSIONS:
    gs = gold_standard[dim_name]
    n_pos = gs.sum()
    print(f"{DISPLAY_NAMES[dim_name]:>22s}:  N_pos={n_pos:5d} / {len(gs)}  "
          f"({100*n_pos/len(gs):.1f}%)")
```

```
Unique texts: 8795
Coders: ['coudry', 'grabsch', 'richter', 'riedel', 'schadt']
          Anti-Elitism:  N_pos= 1340 / 8795  (15.2%)
       People-Centrism:  N_pos=  281 / 8795  (3.2%)
    Left-Wing Ideology:  N_pos=  384 / 8795  (4.4%)
   Right-Wing Ideology:  N_pos=  173 / 8795  (2.0%)
```

4 — Expert-Coder Agreement (PopBERT baseline)

```
popbert_baseline = pd.DataFrame([
    {"Label": "Anti-Elitism",       "N": 3236, "fleiss_kappa": 0.410,
     "agree": 65.80, "F1": 0.496},
    {"Label": "People-Centrism",    "N": 1608, "fleiss_kappa": 0.244,
     "agree": 81.80, "F1": 0.262},
    {"Label": "Left-Wing Ideology", "N": 1393, "fleiss_kappa": 0.355,
     "agree": 84.50, "F1": 0.378},
    {"Label": "Right-Wing Ideology","N":  773, "fleiss_kappa": 0.364,
     "agree": 91.60, "F1": 0.383},
```

```
])
popbert_baseline
```

	Label	N	fleiss_kappa	agree	F1
0	Anti-Elitism	3236	0.410	65.8	0.496
1	People-Centrism	1608	0.244	81.8	0.262
2	Left-Wing Ideology	1393	0.355	84.5	0.378
3	Right-Wing Ideology	773	0.364	91.6	0.383

4a — Recompute expert-coder metrics from raw data

```
def pairwise_f1(mat_df):
    scores = []
    for i, j in combinations(mat_df.columns, 2):
        mask = mat_df[[i, j]].notna().all(axis=1)
        if mask.sum() == 0:
            continue
        scores.append(f1_score(
            mat_df.loc[mask, i].astype(int),
            mat_df.loc[mask, j].astype(int),
            average="binary",
        ))
    return np.mean(scores) if scores else np.nan


def compute_agreement_metrics(mat_df):
    mat = mat_df.dropna().to_numpy().astype(int)
    table, _ = aggregate_raters(mat)
    kappa = fleiss_kappa(table, method="fleiss")
    agree = np.mean(mat.std(axis=1) == 0) * 100
    f1 = pairwise_f1(mat_df.dropna())
    return round(kappa, 3), round(agree, 2), round(f1, 3)


print(f"{'Dimension':>22s}  {'κ':>7s}  {'Agree%':>7s}  {'F1':>6s}")
print("-" * 50)
for dim_name in LABEL_DIMENSIONS:
    kappa, agree, f1 = compute_agreement_metrics(gold_matrices[dim_name])
    print(f"{DISPLAY_NAMES[dim_name]:>22s}  {kappa:7.3f}  {agree:6.2f}%
{f1:.3f}")
```

```
             Dimension        κ   Agree%      F1
--------------------------------------------------
          Anti-Elitism    0.410   65.79%  0.496
```

```
       People-Centrism    0.244   81.84%  0.262
    Left-Wing Ideology    0.355   84.47%  0.378
   Right-Wing Ideology    0.364   91.59%  0.383
```

5 — Load System Prompt & Configure LLM API

Prompt is sent directly as user message concatenated with the text. The model returns JSON with `people_score`, `elite_score`, `antagonism_score` plus explanation fields. We extract and binarize: - `people_score > 0` ⟶ People-Centrism = 1 - `elite_score < 0` ⟶ Anti-Elitism = 1

```
with open("prompt-populism.md", "r", encoding="utf-8") as f:
    PROMPT_RAW = f.read()

print(f"Prompt length: {len(PROMPT_RAW)} chars")
print(PROMPT_RAW[:500], "\n...")
```

```
Prompt length: 8921 chars
You are a political sociology analyst trained to detect populist rhetorical
strategies in social media posts. Your goal is to identify which posts contain
dimensions of populist attitudes — without judging political views.

Analyze each post holistically. Avoid speculation. First, provide a holistic
analysis. Then, evaluate each attitude individually.

---------------------------------------------------------------------------

### CRITICAL INSTRUCTION: THE "NULL" CHECK
Before analyzing, look stric
...
```

```
API_BASE_URL  = "https://llm.scads.ai/v1"
API_KEY       = open(os.path.join("/home/thhaase/Documents/
synosys_internship", "scadsaillm.txt"), "r").readline().strip()
MODEL_NAME    = "Qwen/Qwen3-Coder-30B-A3B-Instruct"
SLEEP_SECONDS = 2
BATCH_SAVE_EVERY = 50

client = OpenAI(base_url=API_BASE_URL, api_key=API_KEY)
```

```
def build_messages(text: str) -> list[dict]:
    """Concatenate prompt + text as a single user message."""
    return [
        {"role": "user", "content": PROMPT_RAW + "\n\n" + text}
    ]
```

```python
def parse_llm_response(answer: str) -> dict | None:
    """Extract JSON object from LLM response."""
    answer = answer.strip()
    if answer.startswith("```"):
        answer = answer.split("\n", 1)[-1].rsplit("```", 1)[0]
    try:
        start = answer.index("{")
        end   = answer.rindex("}") + 1
        return json.loads(answer[start:end])
    except (json.JSONDecodeError, ValueError):
        return None

SCORE_RENAME = {
    "populist_score":  "people_score",
    "elitist_score":   "elite_score",
    "intensity_score":  "antagonism_score",
}

def relabel_keys(parsed: dict) -> dict:
    """Rename LLM output keys to match binarize_response() expectations."""
    return {SCORE_RENAME.get(k, k): v for k, v in parsed.items()}

def binarize_response(parsed: dict) -> dict:
    """
    Convert continuous prompt scores to binary PopBERT labels.
      people_score > 0  →  people_centrism = 1
      elite_score  < 0  →  anti_elitism    = 1
    """
    people = float(parsed.get("people_score", 0))
    elite  = float(parsed.get("elite_score", 0))
    return {
        "anti_elitism":    int(elite < 0),
        "people_centrism": int(people > 0),
        # Also store raw scores for inspection
        "people_score_raw": people,
        "elite_score_raw":  elite,
        "antagonism_score_raw": float(parsed.get("antagonism_score", 0)),
    }
```

6 — Sample 100 Texts & Rate via LLM

Draw a random sample of 100 unique texts, send each to the API, parse and binarize the response. Checkpointed to `llm_predictions.csv`.

```python
SAMPLE_N = 200
```

```python
# Build a joint stratification frame from gold-standard labels
strat = pd.DataFrame({
    dim: gold_standard[dim] for dim in LLM_DIMENSIONS
})

# Create a combined stratum key (e.g. "0_1", "1_0", …)
strat["stratum"] = (
    strat["anti_elitism"].astype(str) + "_" +
strat["people_centrism"].astype(str)
)

# Compute population proportions per stratum
stratum_props = strat["stratum"].value_counts(normalize=True)
stratum_counts = (stratum_props * SAMPLE_N).round().astype(int)

# Adjust rounding so total equals SAMPLE_N exactly
diff = SAMPLE_N - stratum_counts.sum()
if diff != 0:
    # Add/subtract from the largest stratum
    stratum_counts[stratum_counts.idxmax()] += diff

print("Target stratum counts:")
for s, n in stratum_counts.items():
    ae, pc = s.split("_")
    print(f"  Anti-Elitism={ae}, People-Centrism={pc}: {n}")

# Draw stratified sample
sample_ids = (
    strat.groupby("stratum", group_keys=False)
    .apply(lambda g: g.sample(
        n=min(stratum_counts[g.name], len(g)),
        random_state=42,
    ))
    .index
)

texts_to_rate = texts.loc[sample_ids]
print(f"\nSampled {len(texts_to_rate)} unique texts for rating.")

# Verify: gold-standard distribution in the sample
for dim_name in LLM_DIMENSIONS:
    gs_sample = gold_standard[dim_name].loc[sample_ids]
    print(f"  {DISPLAY_NAMES[dim_name]:>22s}: {gs_sample.sum():3d} positive "
          f"({100*gs_sample.mean():.1f}%)")
```

```
Target stratum counts:
  Anti-Elitism=0, People-Centrism=0: 164
  Anti-Elitism=1, People-Centrism=0: 30
  Anti-Elitism=0, People-Centrism=1: 6
  Anti-Elitism=1, People-Centrism=1: 0

Sampled 200 unique texts for rating.
            Anti-Elitism:  30 positive (15.0%)
         People-Centrism:   6 positive (3.0%)
```

```
CHECKPOINT_PATH = "data/llm_predictions_populism.csv"

if os.path.exists(CHECKPOINT_PATH):
    df_done = pd.read_csv(CHECKPOINT_PATH)
    done_ids = set(df_done[ID_COL])
    print(f"Resuming: {len(done_ids)} already rated.")
else:
    df_done = pd.DataFrame()
    done_ids = set()

todo = texts_to_rate.loc[~texts_to_rate.index.isin(done_ids)]
print(f"Remaining: {len(todo)}")
```

```
Remaining: 200
```

```
results = []
errors = 0

for row in texts_to_rate.reset_index().itertuples():
    prompt = PROMPT_RAW + "\n\n" + row.text

    print(f"Processing {row.Index + 1}/{len(texts_to_rate)} | ID:
{row.id}...")
    time.sleep(2)

    response = client.chat.completions.create(
        messages=[
            {"role": "user", "content": prompt}
        ],
        model=MODEL_NAME
    )

    answer = response.choices[0].message.content

    try:
```

```
        answer_json_str = answer[answer.find("{"):answer.rfind("}") + 1]
        answer_data = json.loads(answer_json_str)

        row_result = {
            "id": row.id,
            **binarize_response(relabel_keys(answer_data))
        }
        results.append(row_result)

    except (json.JSONDecodeError, ValueError) as e:
        print(f"Error parsing JSON for ID {row.id}: {e}")
        results.append({"id": row.id, "error": "JSON Parse Error"})
        errors += 1

    if (row.Index + 1) % BATCH_SAVE_EVERY == 0:
        pd.DataFrame(results).to_csv(CHECKPOINT_PATH, index=False)

df_llm = pd.DataFrame(results)
df_llm.to_csv(CHECKPOINT_PATH, index=False)
print(f"\n✓ Done. {len(df_llm)} predictions saved. Parse errors: {errors}")
df_llm.head()
```

```
Processing 1/200 | ID: 211853...
Processing 2/200 | ID: 301878...
Processing 3/200 | ID: 1225217...
Processing 4/200 | ID: 1236966...
Processing 5/200 | ID: 923965...
Processing 6/200 | ID: 1103558...
Processing 7/200 | ID: 302037...
Processing 8/200 | ID: 550012...
Processing 9/200 | ID: 125354...
Processing 10/200 | ID: 932834...
Processing 11/200 | ID: 954439...
Processing 12/200 | ID: 506039...
Processing 13/200 | ID: 73436...
Processing 14/200 | ID: 1030025...
Processing 15/200 | ID: 1032564...
Processing 16/200 | ID: 168479...
Processing 17/200 | ID: 1037836...
Processing 18/200 | ID: 512541...
Processing 19/200 | ID: 576040...
Processing 20/200 | ID: 84032...
Processing 21/200 | ID: 823555...
Processing 22/200 | ID: 324482...
Processing 23/200 | ID: 506040...
Processing 24/200 | ID: 572156...
Processing 25/200 | ID: 1126759...
```

```
Processing 26/200 | ID: 741086...
Processing 27/200 | ID: 194798...
Processing 28/200 | ID: 512634...
Processing 29/200 | ID: 877283...
Processing 30/200 | ID: 765139...
Processing 31/200 | ID: 961277...
Processing 32/200 | ID: 763396...
Processing 33/200 | ID: 941498...
Processing 34/200 | ID: 888884...
Processing 35/200 | ID: 733409...
Processing 36/200 | ID: 60358...
Processing 37/200 | ID: 37996...
Processing 38/200 | ID: 793392...
Processing 39/200 | ID: 1051893...
Processing 40/200 | ID: 229111...
Processing 41/200 | ID: 962718...
Processing 42/200 | ID: 161839...
Processing 43/200 | ID: 536150...
Processing 44/200 | ID: 407210...
Processing 45/200 | ID: 986506...
Processing 46/200 | ID: 455249...
Processing 47/200 | ID: 471674...
Processing 48/200 | ID: 22507...
Processing 49/200 | ID: 271888...
Processing 50/200 | ID: 1101287...
Processing 51/200 | ID: 825999...
Processing 52/200 | ID: 672146...
Processing 53/200 | ID: 1158646...
Processing 54/200 | ID: 585331...
Processing 55/200 | ID: 747357...
Processing 56/200 | ID: 17072...
Processing 57/200 | ID: 616958...
Processing 58/200 | ID: 1146141...
Processing 59/200 | ID: 618909...
Processing 60/200 | ID: 1213862...
Processing 61/200 | ID: 961703...
Processing 62/200 | ID: 165858...
Processing 63/200 | ID: 1017114...
Processing 64/200 | ID: 427609...
Processing 65/200 | ID: 1203349...
Processing 66/200 | ID: 470312...
Processing 67/200 | ID: 1095312...
Error parsing JSON for ID 1095312: Expecting ',' delimiter: line 2 column 73
(char 74)
Processing 68/200 | ID: 13695...
Processing 69/200 | ID: 372754...
Processing 70/200 | ID: 1123435...
Processing 71/200 | ID: 1141281...
```

```
Processing 72/200 | ID: 597193...
Processing 73/200 | ID: 773087...
Processing 74/200 | ID: 478331...
Processing 75/200 | ID: 1202084...
Processing 76/200 | ID: 191864...
Processing 77/200 | ID: 1114936...
Processing 78/200 | ID: 1127747...
Processing 79/200 | ID: 619169...
Processing 80/200 | ID: 1064726...
Processing 81/200 | ID: 1130387...
Processing 82/200 | ID: 721559...
Processing 83/200 | ID: 1201849...
Processing 84/200 | ID: 89693...
Processing 85/200 | ID: 959570...
Processing 86/200 | ID: 40035...
Processing 87/200 | ID: 250525...
Processing 88/200 | ID: 807674...
Processing 89/200 | ID: 619306...
Processing 90/200 | ID: 993952...
Processing 91/200 | ID: 957691...
Processing 92/200 | ID: 749746...
Processing 93/200 | ID: 355271...
Processing 94/200 | ID: 952090...
Processing 95/200 | ID: 427998...
Processing 96/200 | ID: 1137611...
Processing 97/200 | ID: 53824...
Processing 98/200 | ID: 29966...
Processing 99/200 | ID: 285431...
Processing 100/200 | ID: 67582...
Processing 101/200 | ID: 665795...
Processing 102/200 | ID: 52809...
Processing 103/200 | ID: 877992...
Processing 104/200 | ID: 882825...
Processing 105/200 | ID: 14868...
Processing 106/200 | ID: 608464...
Processing 107/200 | ID: 291712...
Processing 108/200 | ID: 846787...
Processing 109/200 | ID: 1110879...
Processing 110/200 | ID: 896097...
Processing 111/200 | ID: 250066...
Processing 112/200 | ID: 61957...
Processing 113/200 | ID: 1147834...
Processing 114/200 | ID: 701791...
Processing 115/200 | ID: 405959...
Processing 116/200 | ID: 374961...
Processing 117/200 | ID: 528354...
Processing 118/200 | ID: 1085068...
Processing 119/200 | ID: 1256331...
```

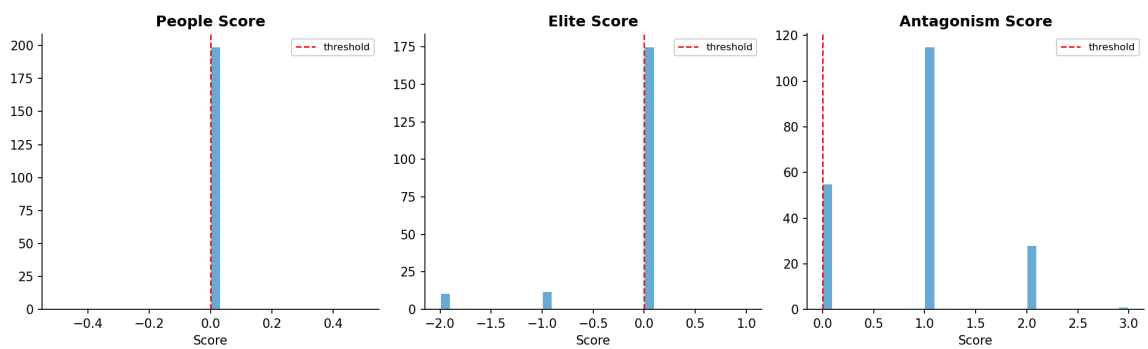
```
Processing 120/200 | ID: 1136588...
Processing 121/200 | ID: 376657...
Processing 122/200 | ID: 1140686...
Processing 123/200 | ID: 399378...
Processing 124/200 | ID: 1211627...
Processing 125/200 | ID: 305964...
Processing 126/200 | ID: 29107...
Processing 127/200 | ID: 232657...
Processing 128/200 | ID: 203003...
Processing 129/200 | ID: 257530...
Processing 130/200 | ID: 933986...
Processing 131/200 | ID: 281749...
Processing 132/200 | ID: 68077...
Processing 133/200 | ID: 1126049...
Processing 134/200 | ID: 354581...
Processing 135/200 | ID: 1088665...
Processing 136/200 | ID: 1022625...
Processing 137/200 | ID: 242414...
Processing 138/200 | ID: 16331...
Processing 139/200 | ID: 625919...
Processing 140/200 | ID: 1200199...
Processing 141/200 | ID: 1033684...
Processing 142/200 | ID: 760432...
Processing 143/200 | ID: 624925...
Processing 144/200 | ID: 674027...
Processing 145/200 | ID: 117779...
Processing 146/200 | ID: 324766...
Processing 147/200 | ID: 875872...
Processing 148/200 | ID: 1069102...
Processing 149/200 | ID: 74721...
Processing 150/200 | ID: 722404...
Processing 151/200 | ID: 78413...
Processing 152/200 | ID: 1042966...
Processing 153/200 | ID: 853652...
Processing 154/200 | ID: 49728...
Processing 155/200 | ID: 967321...
Processing 156/200 | ID: 782204...
Processing 157/200 | ID: 1064236...
Processing 158/200 | ID: 390668...
Processing 159/200 | ID: 105157...
Processing 160/200 | ID: 1043595...
Processing 161/200 | ID: 619515...
Processing 162/200 | ID: 1163603...
Processing 163/200 | ID: 672324...
Processing 164/200 | ID: 603129...
Processing 165/200 | ID: 99020...
Processing 166/200 | ID: 597860...
Processing 167/200 | ID: 331065...
```

```
Processing 168/200 | ID: 661882...
Processing 169/200 | ID: 1017437...
Processing 170/200 | ID: 800088...
Processing 171/200 | ID: 973692...
Processing 172/200 | ID: 93877...
Processing 173/200 | ID: 448692...
Processing 174/200 | ID: 134551...
Processing 175/200 | ID: 44913...
Processing 176/200 | ID: 1134921...
Processing 177/200 | ID: 1099999...
Processing 178/200 | ID: 894107...
Processing 179/200 | ID: 213821...
Processing 180/200 | ID: 245172...
Processing 181/200 | ID: 1099259...
Processing 182/200 | ID: 324476...
Processing 183/200 | ID: 494590...
Processing 184/200 | ID: 32682...
Processing 185/200 | ID: 57889...
Processing 186/200 | ID: 986547...
Processing 187/200 | ID: 158090...
Processing 188/200 | ID: 954653...
Processing 189/200 | ID: 914735...
Processing 190/200 | ID: 195396...
Processing 191/200 | ID: 909399...
Processing 192/200 | ID: 782942...
Processing 193/200 | ID: 302912...
Processing 194/200 | ID: 1073239...
Processing 195/200 | ID: 858617...
Processing 196/200 | ID: 924397...
Processing 197/200 | ID: 468672...
Processing 198/200 | ID: 468056...
Processing 199/200 | ID: 486292...
Processing 200/200 | ID: 1030730...

✓ Done. 200 predictions saved. Parse errors: 1
```

	id	anti_elitism	people_centrism	people_score_raw	elite_score_raw	antagonism_score_raw	error
0	211853	0.0	0.0	0.0	0.0	1.0	NaN
1	301878	0.0	0.0	0.0	0.0	1.0	NaN
2	1225217	0.0	0.0	0.0	0.0	0.0	NaN
3	1236966	0.0	0.0	0.0	0.0	0.0	NaN
4	923965	0.0	0.0	0.0	0.0	1.0	NaN

6a — Inspect raw score distributions

```
fig, axes = plt.subplots(1, 3, figsize=(13, 4))
for ax, col, title in zip(axes,
    ["people_score_raw", "elite_score_raw", "antagonism_score_raw"],
    ["People Score", "Elite Score", "Antagonism Score"]):
    vals = df_llm[col].dropna()
    ax.hist(vals, bins=30, color="#6baed6", edgecolor="white")
    ax.axvline(0, color="red", ls="--", lw=1.2, label="threshold")
    ax.set_title(title, fontweight="bold")
    ax.set_xlabel("Score")
    ax.legend(fontsize=8)
    ax.spines[["top", "right"]].set_visible(False)
fig.tight_layout()
plt.show()

print(f"\nBinarized label counts (n={len(df_llm)}):")
for dim in LLM_DIMENSIONS:
    n1 = df_llm[dim].sum()
    print(f"  {DISPLAY_NAMES[dim]:>22s}: {int(n1):3d} positive ({100*n1/
len(df_llm):.1f}%)")
```




```
Binarized label counts (n=200):
            Anti-Elitism:  23 positive (11.5%)
         People-Centrism:   0 positive (0.0%)
```

7 — Compute LLM vs. Gold-Standard Metrics (sample-based estimate)

Computed on the 200-text sample with bootstrap 95% CIs. Only Anti-Elitism and People-Centrism are evaluated (prompt scope).

```
if 'df_llm' not in dir():
    df_llm = pd.read_csv("data/llm_predictions2.csv")
```

```python
def bootstrap_ci(y_true, y_pred, metric_fn, n_boot=1000, ci=0.95):
    rng = np.random.RandomState(42)
    scores = []
    n = len(y_true)
    for _ in range(n_boot):
        idx = rng.choice(n, n, replace=True)
        scores.append(metric_fn(y_true[idx], y_pred[idx]))
    lo = np.percentile(scores, (1 - ci) / 2 * 100)
    hi = np.percentile(scores, (1 + ci) / 2 * 100)
    return lo, hi


llm_metrics = []

for dim_name in LLM_DIMENSIONS:
    gs  = gold_standard[dim_name]
    llm = df_llm.set_index(ID_COL)[dim_name]

    common = gs.index.intersection(llm.dropna().index)
    y_true = gs.loc[common].astype(int).values
    y_pred = llm.loc[common].astype(int).values

    n = len(common)
    n_pos = int(y_pred.sum())

    f1_val = f1_score(y_true, y_pred, average="binary", zero_division=0)
    kappa  = cohen_kappa_score(y_true, y_pred)
    agree  = np.mean(y_true == y_pred) * 100

    f1_lo, f1_hi = bootstrap_ci(
        y_true, y_pred,
        lambda yt, yp: f1_score(yt, yp, average="binary", zero_division=0))

    # Pairwise F1: LLM vs. each individual coder
    mat = gold_matrices[dim_name]
    pw_scores = []
    for coder in mat.columns:
        idx = common.intersection(mat[coder].dropna().index)
        if len(idx) > 0:
            pw_scores.append(f1_score(
                mat[coder].loc[idx].astype(int),
                llm.loc[idx].astype(int),
                average="binary", zero_division=0,
            ))
    f1_vs_coders = np.mean(pw_scores) if pw_scores else np.nan

    llm_metrics.append({
```

```
        "Label":        DISPLAY_NAMES[dim_name],
        "N":            n,
        "N_pos":        n_pos,
        "kappa":        round(kappa, 3),
        "agree":        round(agree, 2),
        "F1_vs_gold":   round(f1_val, 3),
        "F1_95ci":      f"[{f1_lo:.3f}, {f1_hi:.3f}]",
        "F1_vs_coders": round(f1_vs_coders, 3),
    })

df_metrics = pd.DataFrame(llm_metrics)
df_metrics
```

	Label	N	N_pos	kappa	agree	F1_vs_gold	F1_95ci	F1_vs_coders
0	Anti-Elitism	199	23	0.327	84.42	0.415	[0.238, 0.576]	0.368
1	People-Centrism	199	0	0.000	96.98	0.000	[0.000, 0.000]	0.000

8 — Combined Comparison Table

```
# Expert baseline — only the two dimensions our prompt covers
tbl_expert = popbert_baseline[
    popbert_baseline["Label"].isin(["Anti-Elitism", "People-Centrism"])
].rename(columns={"fleiss_kappa": "κ", "agree": "Agreement %"}).copy()
tbl_expert["Source"] = "Expert Coders"

# LLM results
tbl_llm = df_metrics[["Label", "N", "kappa", "agree", "F1_vs_gold"]].copy()
tbl_llm.columns = ["Label", "N", "κ", "Agreement %", "F1"]
tbl_llm["Source"] = "LLM Prompt (n=200)"

# Add mean rows
rows = []
for tbl in [tbl_expert, tbl_llm]:
    mean_row = pd.DataFrame([{
        "Label": "Mean",
        "N": "",
        "κ": round(tbl["κ"].mean(), 3),
        "Agreement %": round(tbl["Agreement %"].mean(), 2),
        "F1": round(tbl["F1"].mean(), 3),
        "Source": tbl["Source"].iloc[0],
    }])
    rows.append(pd.concat([tbl, mean_row], ignore_index=True))
```

```
combined = pd.concat(rows, ignore_index=True)
combined = combined[["Source", "Label", "N", "κ", "Agreement %", "F1"]]
print(combined.to_string(index=False))
combined
```

```
            Source           Label    N     κ  Agreement %    F1
     Expert Coders    Anti-Elitism 3236 0.410        65.80 0.496
     Expert Coders People-Centrism 1608 0.244        81.80 0.262
     Expert Coders            Mean      0.327        73.80 0.379
LLM Prompt (n=200)    Anti-Elitism  199 0.327        84.42 0.415
LLM Prompt (n=200) People-Centrism  199 0.000        96.98 0.000
LLM Prompt (n=200)            Mean      0.164        90.70 0.208
```

	Source	Label	N	κ	Agreement %	F1
0	Expert Coders	Anti-Elitism	3236	0.410	65.80	0.496
1	Expert Coders	People-Centrism	1608	0.244	81.80	0.262
2	Expert Coders	Mean		0.327	73.80	0.379
3	LLM Prompt (n=200)	Anti-Elitism	199	0.327	84.42	0.415
4	LLM Prompt (n=200)	People-Centrism	199	0.000	96.98	0.000
5	LLM Prompt (n=200)	Mean		0.164	90.70	0.208

```
combined.to_latex("images/comparison_table2.tex", index=False, escape=False,
                  float_format="%.3f")
print("Saved: comparison_table.tex")
```

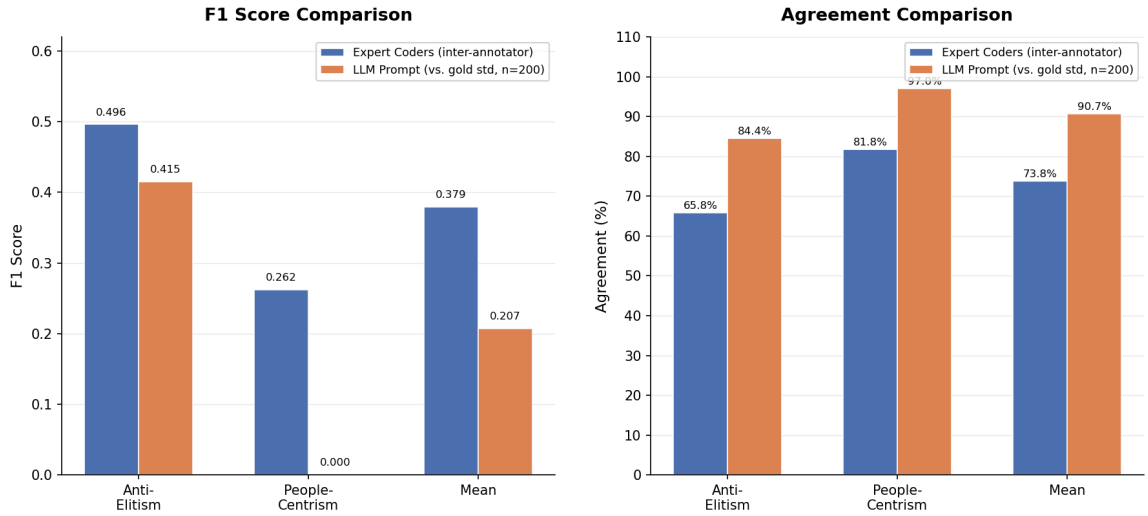
9 — Bar Plot: Expert Coders vs. LLM Prompt

Grouped bar plot for the two dimensions the prompt covers: Anti-Elitism and People-Centrism.

```
labels = ["Anti-\nElitism", "People-\nCentrism", "Mean"]

# Expert values (Anti-Elitism, People-Centrism only)
pb = popbert_baseline[popbert_baseline["Label"].isin(
    ["Anti-Elitism", "People-Centrism"])].reset_index(drop=True)
expert_f1    = list(pb["F1"]) + [pb["F1"].mean()]
expert_agree = list(pb["agree"]) + [pb["agree"].mean()]

llm_f1    = list(df_metrics["F1_vs_gold"]) + [df_metrics["F1_vs_gold"].mean()]
llm_agree = list(df_metrics["agree"])      + [df_metrics["agree"].mean()]

x = np.arange(len(labels))
w = 0.32
```

```
c_expert = "#4C72B0"
c_llm    = "#DD8452"

fig, axes = plt.subplots(1, 2, figsize=(12, 5.5))

# ── F1 ──────────────────────────────────────────────────────────────
ax = axes[0]
b1 = ax.bar(x - w/2, expert_f1, w, label="Expert Coders (inter-annotator)",
            color=c_expert, edgecolor="white", linewidth=0.8)
b2 = ax.bar(x + w/2, llm_f1,    w, label="LLM Prompt (vs. gold std, n=200)",
            color=c_llm, edgecolor="white", linewidth=0.8)
for bg in [b1, b2]:
    for bar in bg:
        h = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2, h + 0.01, f"{h:.3f}",
                ha="center", va="bottom", fontsize=8, fontweight="medium")
ax.set_ylabel("F1 Score", fontsize=11)
ax.set_title("F1 Score Comparison", fontsize=13, fontweight="bold", pad=12)
ax.set_xticks(x); ax.set_xticklabels(labels, fontsize=10)
ax.set_ylim(0, max(max(expert_f1), max(llm_f1)) * 1.25)
ax.legend(fontsize=8.5, loc="upper right")
ax.spines[["top", "right"]].set_visible(False)
ax.yaxis.set_major_locator(mticker.MultipleLocator(0.1))
ax.grid(axis="y", alpha=0.3, linewidth=0.5); ax.set_axisbelow(True)

# ── Agreement ───────────────────────────────────────────────────────
ax = axes[1]
b1 = ax.bar(x - w/2, expert_agree, w, label="Expert Coders (inter-annotator)",
            color=c_expert, edgecolor="white", linewidth=0.8)
b2 = ax.bar(x + w/2, llm_agree,    w, label="LLM Prompt (vs. gold std,
n=200)",
            color=c_llm, edgecolor="white", linewidth=0.8)
for bg in [b1, b2]:
    for bar in bg:
        h = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2, h + 0.5, f"{h:.1f}%",
                ha="center", va="bottom", fontsize=8, fontweight="medium")
ax.set_ylabel("Agreement (%)", fontsize=11)
ax.set_title("Agreement Comparison", fontsize=13, fontweight="bold", pad=12)
ax.set_xticks(x); ax.set_xticklabels(labels, fontsize=10)
ax.set_ylim(0, 110)
ax.legend(fontsize=8.5, loc="upper right")
ax.spines[["top", "right"]].set_visible(False)
ax.yaxis.set_major_locator(mticker.MultipleLocator(10))
ax.grid(axis="y", alpha=0.3, linewidth=0.5); ax.set_axisbelow(True)

fig.tight_layout(w_pad=3)
fig.savefig("images/comparison_barplot.png", bbox_inches="tight")
```

```
fig.savefig("images/comparison_barplot.pdf", bbox_inches="tight")
plt.show()
print("Saved: comparison_barplot.png / .pdf")
```

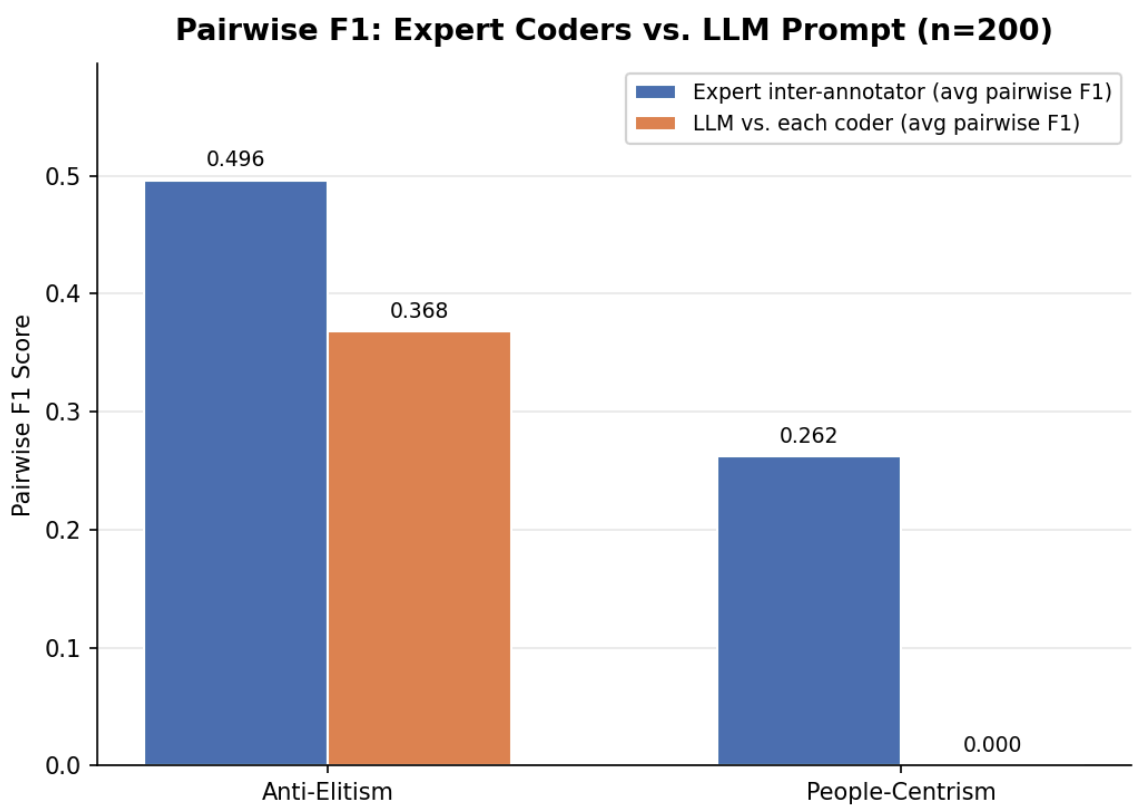



```
Saved: comparison_barplot.png / .pdf
```

10 — Bonus: LLM vs. Individual Coders (Pairwise F1)

```
fig, ax = plt.subplots(figsize=(7, 5))

labels_short = [DISPLAY_NAMES[d] for d in LLM_DIMENSIONS]
x = np.arange(len(labels_short))
w = 0.32
c_expert = "#4C72B0"
c_llm    = "#DD8452"

expert_pw = [pb.loc[pb["Label"] == DISPLAY_NAMES[d], "F1"].values[0]
             for d in LLM_DIMENSIONS]
llm_pw    = list(df_metrics["F1_vs_coders"])

b1 = ax.bar(x - w/2, expert_pw, w, label="Expert inter-annotator (avg pairwise
F1)",
            color=c_expert, edgecolor="white")
b2 = ax.bar(x + w/2, llm_pw,    w, label="LLM vs. each coder (avg pairwise
F1)",
            color=c_llm, edgecolor="white")
for bg in [b1, b2]:
    for bar in bg:
        h = bar.get_height()
```

```
        ax.text(bar.get_x() + bar.get_width()/2, h + 0.008, f"{h:.3f}",
                ha="center", va="bottom", fontsize=9)

ax.set_ylabel("Pairwise F1 Score")
ax.set_title("Pairwise F1: Expert Coders vs. LLM Prompt (n=200)",
             fontsize=13, fontweight="bold", pad=10)
ax.set_xticks(x); ax.set_xticklabels(labels_short, fontsize=10)
ax.set_ylim(0, max(max(expert_pw), max(llm_pw)) * 1.2)
ax.legend(fontsize=9)
ax.spines[["top", "right"]].set_visible(False)
ax.grid(axis="y", alpha=0.3); ax.set_axisbelow(True)

fig.tight_layout()
fig.savefig("images/pairwise_f1_barplot.png", bbox_inches="tight")
plt.show()
print("Saved: pairwise_f1_barplot.png")
```




```
Saved: pairwise_f1_barplot.png
```